# ENSF 380

## TOPIC 13: EXCEPTION HANDLING

# Types of Exceptions

Error
- Unexpected
- External

Runtime exceptions
- Unexpected
- Internal

**Checked exceptions**

# Why Use Exceptions

Interrupt a program to prevent unexpected results

Provide a structured handling separate from regular code

Provide a trace of the error

Grouping and differentiating error types

```
System.exit(1);
```

# Using Exceptions

IllegalArgumentException

CloneNotSupportedException

IndexOutOfBoundsException

```
public String arrayConcat(int start) throws IndexOutofBoundsException {



if ((start >= this.size()) || (start < 0)) {
  throw new IndexOutOfBoundsException("Index " + start + " is out of bounds");
}
```

# Using Exceptions



```
class ExampleObject {
    private int storedValue;
    public ExampleObject(int value) throws Exception {
        this.setValue(value);
    }

    public void setValue(int value) throws Exception {
        if (value < 0) {
            throw new Exception("Value must be zero or greater");
        }
        this.storedValue = value;
    }
}
```

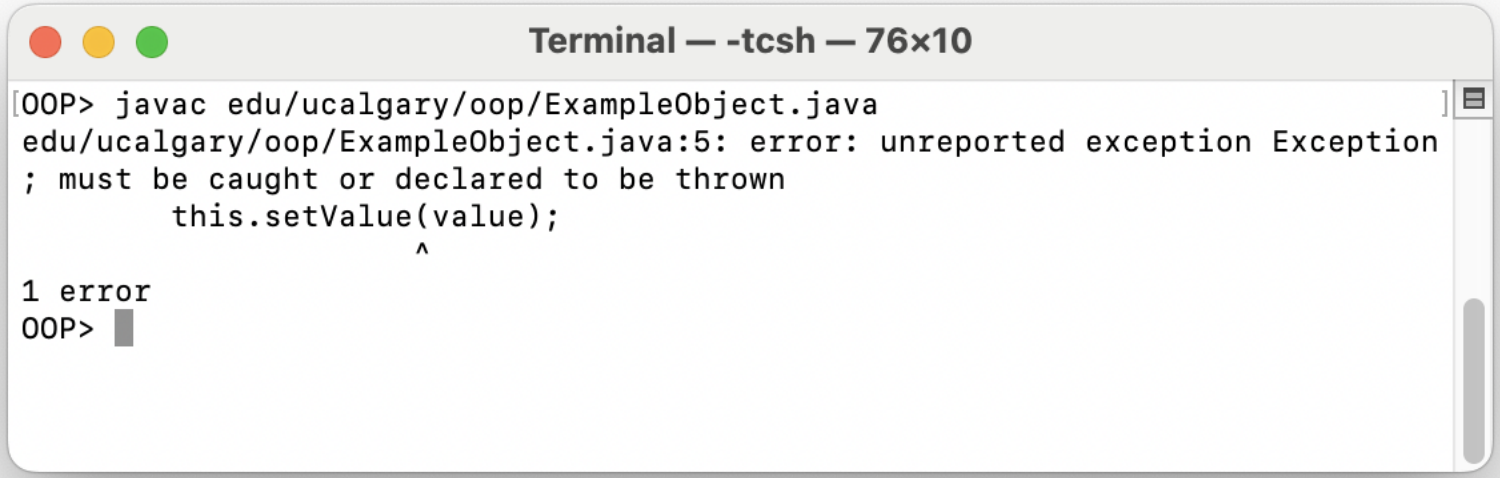
# Using Exceptions



```
package edu.ucalgary.oop;

class ExampleObject {
    private int storedValue;
    public ExampleObject(int value) throws Exception {
        this.setValue(value);
    }

    public void setValue(int va
        if (value < 0) {
            throw new Exception
        }
        this.storedValue = valu
    }
}
```



# Using Exceptions



```
package edu.ucalgary.oop;



public class Main {
  public static void main(String[] args) throws Exception {
      ExampleObject myObject = new ExampleObject(4);
   }
}
```

# Exercise 17.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

# User-Defined Exceptions

02_UserDefined

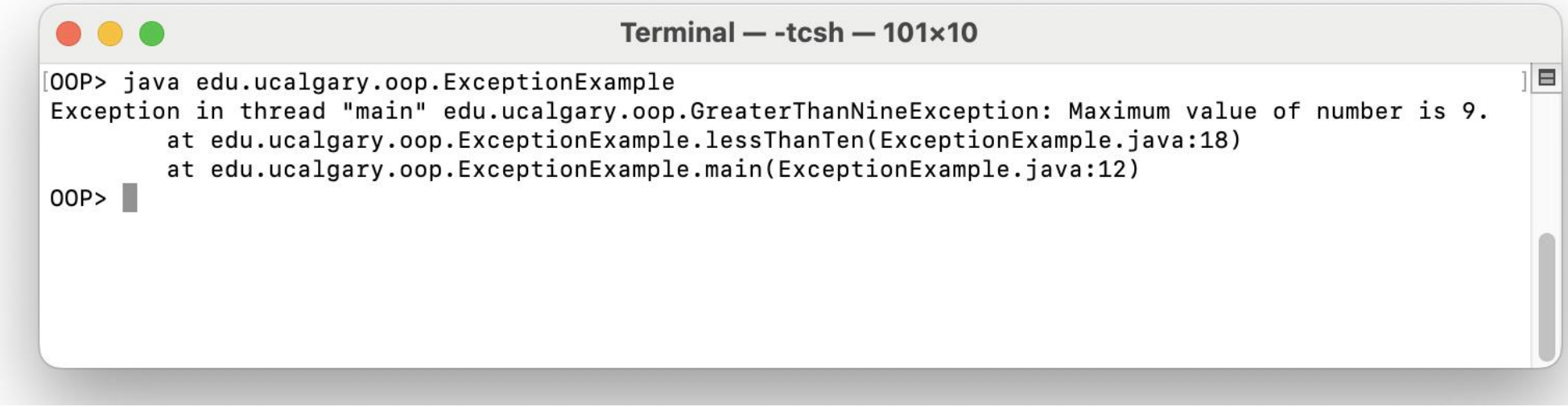
```
public class ExceptionExample {
	static public void main (String[] args) throws GreaterThanNineException {
		var example = new ExceptionExample();
		example.lessThanTen(11);
	}
	public void lessThanTen(int number) throws GreaterThanNineException {
		if (number >= 10) {
			throw new GreaterThanNineException();
		}
	}
}
class GreaterThanNineException extends Exception {
		public GreaterThanNineException() {
			super("Maximum value of number is 9.");
		}
}
```

# User-Defined Exceptions

02_UserDefined

```
public class ExceptionExample {
    static public void main (String[] args) throws GreaterThanNineException {
        var example = new ExceptionExample();
        example.lessThanTen(11);
    }
    public void lessThanTen(int number) throws GreaterThanNineException {
        if (number >= 10) {
            throw new GreaterThanNineException();
        }
    }
}
class GreaterTha
        public G

        }
}
```



# Exercise 17.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

# Exception Catching

Surrounding block of code

Calling code

JVM

# Catch

03_Catch

```
...
public int divide(int number, int divisor) {
  int answer;

  try {
    answer = number / divisor;
  }

  catch(ArithmeticException e) {
    System.out.println("Caught an arithmetic error " + e.getMessage());
    return 100;
  }
}
...
```

# Catch

03_Catch

```
...

public int divide(int number, int divisor) {
  int answer;

  try {
    answer = number / divisor;
  }

  catch(ArithmeticException e) {
    System.out.println("Caught an arithmetic error " + e.getMessage());
    return 100;
  }
}

...
```

**Review the documentation for Throwable.**

# Catch

03_Catch

```
try {
  answer = number / divisor;
}

catch(ArithmeticException e) {
  System.out.println("Caught an arithmetic error " + e.getMessage());
  return 100;
}

catch(Exception e) {
  System.out.println("Caught an exception: " + e.getMessage());
  return 200;
}
```

# Catch

03_Catch

```
...

finally {
  System.out.println("The finally block was executed.");
  return 300;
}

...
```

# Catch

04_CallerCatch

```
static public void main (String[] args) {
  var example = new ExceptionExample();
  int answer = 200;

  try {
    answer = example.divide(5, 0);
  }
  catch(ArithmeticException e) {
    System.out.println("Caught an arithmetic error " + e.getMessage());
  }

  System.out.println("Answer was: " + answer);
}

public int divide(int number, int divisor) {
  int answer = number / divisor;
  return answer;
}
```

# Exercise 17.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

# Summary: Knowledge

Types of exceptions

Why to use exceptions

The call stack for catching exceptions

The ordering of catch blocks

# Summary: Skills

Using a try block

Using a catch block

Using the finally block

Writing user-defined exceptions

Exiting a program with System.exit()

# Additional Information

Related reading

- Volume 1, Chapter 7.1: Dealing with Errors
- Volume 1, Chapter 7.2: Catching Exceptions
- Volume 1, Chapter 7.3: Tips for Using Exceptions